

GPters 22기

코드화 패턴

비개발자가 AI를 다루는 새로운 패러다임

DY Cho (조셉)

오프닝

"의자 = 방패와 창"

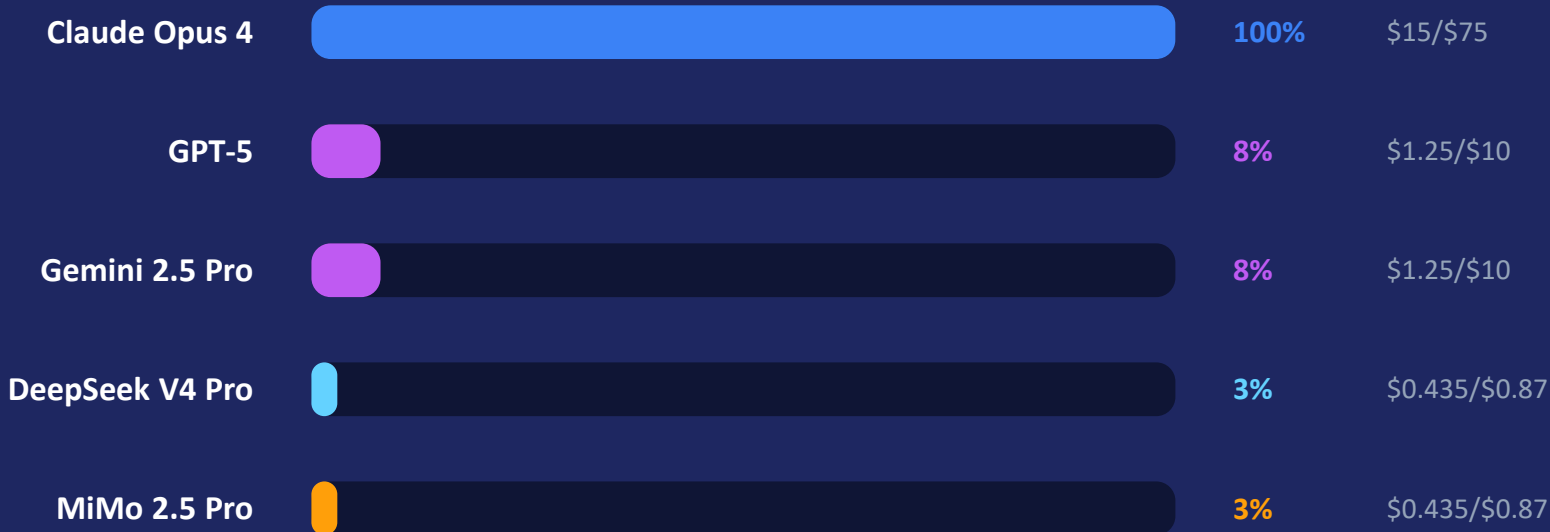
Parker(2013) 영화에서 의자는 단순한 가구가 아니었다.
방어의 방패이자 공격의 창이었다.

저는 AI를 그냥 '시켜 먹는' 도구로 썼습니다.

그러다 \$100이 하루만에 사라졌습니다.

문제 정의

MD 강제성이 모델별로 달라서 토큰비 낭비가 커진다



숨겨진 비용: Thinking 토큰

Input: 사용자 프롬프트 → input 가격 Output: 모델 응답 → output 가격

Thinking (추론): 모델의 내부 추론 → output 가격으로 과금 (input 아님!)

Thinking 포함 (\$0.27/회)

입력 1,000 + thinking 10,000 + 출력 500 토큰

\$100으로 약 374번 대화

이런 작업에 thinking 발생:
분석 · 기획 · 추론 · 창작

Thinking 없이 (\$0.02/회)

입력 1,000 + 출력 500 토큰

\$100으로 약 5,714번 대화

이런 작업은 thinking 없음:
웁기기 · 분류 · 설명 · 생성

왜 코드화인가?

3가지 이익

97%

토큰비 절감

Thinking 토큰 비용 0

100%

검증 일관성

모델마다 다른 해석 → 통일



모델 변경 무관

새 모델이 나와도 유지

코드화 =

AI가 반복 검증할 "입력값"을 만드는 것

= 검증 툴체인을 구축하는 것

= 비개발자의 시간·에너지를 절약하면서 AI의 판단력은 유지하는 것

최종값 패턴 ✕

PDF 규칙



Python 코드



통과/위반

최종 판정 (AI 판단 없음, 경직됨)

상황 맥락 무시, 예외 판단 불가

입력값 패턴 ✓

PDF 규칙



Python 코드



입력값



AI 판단

상황 맥락 고려, 예외 판단 가능

비개발자가 "절대적"이기 전의 약한 채도 된

코드화의 4가지 전제 조건

공조검규

공식화

이 계산으로 결과를 냄

키워드 밀도 = 키워드 수 / 전체 단어 수 × 100

조건화

이 조건이면 통과, 아니면 위반

키워드 3개 이상이면 통과

검증화

맞다/틀리다로 판별할 수 있어야 함

검증할 수 없는 것(감정, 창의성)은 코드화 불가

구상화

이 형식이어야 함

실전 사례

marketing_rules.py

PDF 6권
마케팅 전자책



AI가 추출
조건·공식·규격화



비개발자 검토
O/X 판단만



코드화
Python 파일



검증 자동화
반복 실행

100X

토큰비 절감

6권

PDF 규칙 코드화

301줄

Python 코드

실전 사례

파일 의존성 구조

Before

메모리 (87%)

규칙 전체 저장 (13,151자)

모델마다 해석 다름
중복 관리



After

메모리 (55%)

"참조 한 줄"



Python 파일 (코드화)

모델 독립성 확보

결론

"AI를 도구로 소비하지 말고
시스템으로 설계하라"

쉽게 말하면,

'시켜 먹지 말고 같이 일하라'는 겁니다.

비개발자가 "시스템을 설계"하는 경험